МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ІВАНА ФРАНКА

ЗВІТ 1

з дисципліни “Функціональне програмування”

на тему « Основи функціонального програмування у Python »

Виконав:

Студент 2 курсу

групи ФеП-23

МАГОРА Максим

Перевірив:

СВАЛЕБА Сергій Андрійович

ЗВІТ № 1

з дисципліни «Функціональне програмування»

Тема: Основи функціонального програмування у Python

PDFi ще 1

Варіант № 11: Мобільний тариф

PDFi ще 1

Виконав: Студент 2 курсу групи ФеП-23 МАГОРА Максим

PDF

Перевірив: СВАЛЕБА Сергій Андрійович

PDF

Структура програмного проєкту

Для відповідності вимогам автоматичної перевірки використаємо таку структуру:

Plain Text

```
lab01/
│
├── README.md
├── core.py
├── app.py
├── requirements.txt
│
└── tests/
    └── test_core.py
```

У програмі будуть реалізовані функції `calculate_minutes_cost`, `calculate_sms_cost`, `calculate_traffic_cost`, `calculate_monthly_cost`, `with_monthly_cost`, `filter_tariffs` і `make_processor`.

Ключові слова `mobile`, `tariff`, `minutes`, `sms`, `traffic`, `data`, `bonus`, `monthly`, `cost`, `limit`, `filter` будуть використані в назвах функцій, параметрів та логіці програми.

Для прикладу приймемо такі навчальні умови тарифікації: абонентська плата — 150 грн, пакет — 100 хвилин, 50 SMS та 10 ГБ інтернету. Перевищення пакетних лімітів оплачується окремо. Це демонстраційні параметри, а не тарифи реального оператора.

1. Мета та завдання роботи

- Зрозуміти сутність чистої функції та референтної прозорості.

  DOCX

- Навчитися виявляти та ізолювати побічні ефекти (I/O, мутації, глобальний стан).

  DOCX

- Переписати імперативний фрагмент коду у функціональному стилі.

  DOCX

- Застосувати `typing.Callable` для параметризації обчислень, фільтрації та політик бонусів.

  DOCX

- Інтегрувати специфічні ключові слова варіанта: `mobile`, `tariff`, `minutes`, `sms`, `traffic`, `data`, `bonus`, `monthly`, `cost`, `limit`, `filter`.

  MD

2. Структура проєкту та модулів (Варіант № 11)

Проєкт організовано відповідно до методичних вказівок та розділено на такі модулі та папки:

PDFі ще 2

Plaintext

```
lab01/
├── core.py             # Чисте функціональне ядро (розрахунки та
політики)[cite: 10]
├── app.py              # Оболонка побічних ефектів та I/O[cite: 10]
├── tests/
│   └── test_core.py    # Pytest-тести на референтну прозорість і мутації[cite:
10]
├── requirements.txt    # Залежності проєкту (pytest, mypy, black, ruff)[cite:
10, 14]
└── README.md           # Документація до лабораторної роботи[cite: 10]
```

PDF

3. Опис і назви методів (функцій), реалізованих у проєкті

Для Варіанта 11 («Мобільний тариф») реалізовано такі чисті функції та компоненти:

A. Чисте ядро (`core.py`)

- **`calculate_minutes_cost(minutes: int, limit: int, price: float) -> float`**

  Опис: Чиста функція розрахунку вартості понаднормових хвилин (`minutes`) понад встановлений ліміт (`limit`).

  PY

- **`calculate_sms_cost(sms: int, limit: int, price: float) -> float`**

  Опис: Чиста функція розрахунку вартості понаднормових SMS-повідомлень (`sms`) понад ліміт.

  PY

- **`calculate_traffic_cost(traffic: float, data_limit: float, price: float) -> float`**

  Опис: Чиста функція розрахунку вартості понаднормового інтернет-трафіку/даних (`traffic`, `data`).

  MDi ще 1

- **`calculate_monthly_cost(usage, ...) -> float`**

  Опис: Чиста функція обчислення базової вартості послуг до застосування знижок/бонусів.

  PY

- **`with_monthly_cost(usage: MobileUsage, monthly_cost: float) -> MobileResult`**

  Опис: Функція, що повертає **новий словник** (створює новий об'єкт із полем `monthly_cost`), не мутуючи вхідний об'єкт.

  MDi ще 1

- **`filter_tariffs(usages: list[MobileUsage], accept: FilterFn) -> list[MobileUsage]`**

  Опис: Функція фільтрації тарифів (`tariff`, `filter`, `limit`) за заданим критерієм.

  MDi ще 1

- **`make_processor(*, accept: FilterFn, tariff_policy: TariffFn, bonus_policy: BonusFn) -> Callable[...]`**

  Опис: Функція вищого порядку (фабрика обробника), яка приймає правила фільтрації, тарифікації та політики нарахування бонусів (`bonus`) через `typing.Callable`.

  PY

Б. Оболонка побічних ефектів (`app.py`)

- **`render_report(results: list[MobileResult]) -> None`**

  Опис: Єдина функція виведення (`I/O`, `print`), яка приймає чистий результат роботи ядра та відображає його в консолі.

  PY

- **`main() -> None`**

  Опис: Головна точка входу, що ініціалізує демонстраційні дані абонентів (`mobile`, `tariff`), конфігурує політики та запускає процесор.

  PY

4. Висновки

Під час виконання лабораторної роботи №1 (варіант 11 — «Мобільний тариф») було практично закріплено принципи функціонального програмування на мові Python:

PDF

1. **Забезпечення чистоти функцій та референтної прозорості:** Усі розрахункові функції (`calculate_minutes_cost`, `calculate_sms_cost`, `calculate_traffic_cost`, `calculate_monthly_cost`) є детермінованими, не виконують втручання у глобальний стан і завжди повертають однаковий результат за однакових вхідних параметрів.

   PDFі ще 1

2. **Ізоляція побічних ефектів:** Усі операції введення-виведення (`print`) ізольовані виключно у модулі `app.py`(`render_report`). Використання генерації нових словників у `with_monthly_cost` повністю унеможливлює мутацію вхідних даних.

   PYі ще 2

3. **Параметризація через `typing.Callable`:** Завдяки функції вищого порядку `make_processor` та передачі політик (`tariff_policy`, `bonus_policy`, `accept`) вдалося повністю відокремити процес обробки колекції від бізнес-логіки розрахунків.

   PDFi ще 1

4. **Тестування:** За допомогою `pytest`-тестів (`test_referential_transparency`, `test_no_mutation`, `test_calculations`, `test_callable_policies`, `test_filter_tariffs`) було успішно перевірено детермінованість коду, відсутність побічних ефектів та коректність обчислень згідно з демонстраційними умовами.

   MDi ще 1